Semantic Similarity Search for Unfair Clause Detection

Technical Report

S. N. Thibaud
snthibaud@gmail.com

---

1 Problem & Approach

Terms of Service documents routinely contain clauses that may be unfair to consumers. The CLAUDETTE project [3] provides an annotated corpus of 9,414 sentences from 50 ToS documents (from 12,011 raw sentences after discarding fragments shorter than five words), each labelled along eight unfairness categories. Only 1,032 sentences (11%) carry any unfairness label.

The goal is a production-style service that indexes the corpus, supports semantic similarity search, classifies unfair clauses, and is containerised with Docker. Generative language models and retrieval-augmented generation are out of scope.

Methodology. Each experiment tests a specific hypothesis, produces a quantitative result, and raises the next question. The progression is:

1. **Baseline**: embed with Legal-BERT, classify with Logistic Regression. Is this viable?
2. **Domain**: does legal-domain pre-training matter? Compare against a general-domain SBERT model.
3. **SBERT fine-tuning**: can contrastive fine-tuning of the encoder improve results without changing the architecture?
4. **Significance**: are the observed differences real, or within statistical noise? Bootstrap the ranking metric.
5. **External validation**: how does the system compare to Lippi et al. [3] under their exact evaluation protocol?

All code was developed iteratively with an AI coding assistant (pi) and manually verified: every reported number can be reproduced with the commands in Section 5.

Metrics. Three metric families are used throughout:

MCC Matthews Correlation Coefficient, for N samples and category c:

$$\mathrm{MCC}_c = \frac{\mathrm{TP}_c \cdot \mathrm{TN}_c - \mathrm{FP}_c \cdot \mathrm{FN}_c}{\sqrt{(\mathrm{TP}_c + \mathrm{FP}_c)(\mathrm{TP}_c + \mathrm{FN}_c)(\mathrm{TN}_c + \mathrm{FP}_c)(\mathrm{TN}_c + \mathrm{FN}_c)}}$$

Accounts for all four confusion-matrix quadrants, unbiased by class imbalance ($\sim$11% positive). Used for model selection (C tuning) and threshold calibra-

tion. Requires a chosen threshold. Reported as macro-average across the 8 categories.

AP Average Precision, the area under the precision-recall curve for a ranked list of N predictions with true labels $y_i \in \{0, 1\}$:

$$\text{AP} = \sum_{k=1}^{N} \text{Prec}(k) \cdot \Delta\text{Rec}(k)$$

where $\text{Prec}(k)$ and $\text{Rec}(k)$ are precision and recall at rank k. Threshold-agnostic; measures ranking quality across all operating points. Reported with bootstrap 95% confidence intervals (2,000 paired iterations).

Search metrics Semantic search is evaluated via label overlap as a weak-supervision proxy: two sentences are *relevant* iff they share ≥ 1 positive unfairness label. Let Q be the set of 1,032 query sentences that carry ≥ 1 label (fair sentences carry no evaluation signal). For a query $q \in Q$, let R_q be its top-k retrieved neighbours:

$$\text{Prec@}k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|\{r \in R_q : \text{relevant}(q, r)\}|}{k}$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}, \quad \text{rank}_q = \text{position of first relevant result (0 if none)}$$

$$\text{Gain} = \frac{\text{Prec@}k}{\text{random-Prec@}k}$$

where random-Prec@k uses k uniformly sampled non-query sentences as the baseline. This proxy is a lower bound — sentences can be similar without sharing a label — but it requires no additional annotation.

2 Experiments

2.1 Baseline: Legal-BERT + Logistic Regression

Hypothesis: frozen embeddings from a domain-adapted BERT model, classified by a simple linear model, can capture enough legal semantics for both search and unfair-clause detection.

Setup: encode all 9,414 sentences with `nlpaueb/legal-bert-base-uncased` [1] (110M parameters, 768-d, mean pooling over the last hidden layer), store in a FAISS `IndexFlatIP` [2] with L2-normalised vectors, and train a multi-output Logistic Regression (`class_weight='balanced'`, C tuned via 5-fold CV) on the frozen embeddings. Evaluate on a stratified 85/15 validation split.

Result: Validation MCC = 0.720, macro AP = 0.736. The classifier is viable: per-category MCC ranges from 0.61 (Limitation of Liability) to 0.83 (Unilateral Change—Usage). Precision@10 for semantic search is 0.565, a 24.6$\times$ gain over random.

Next question: how much does the legal-domain pre-training contribute?

2.2 Domain pre-training: Legal-BERT vs. all-MiniLM

Hypothesis: domain-specific pre-training provides a measurable advantage over a general-purpose embedding model.

Setup: replace the encoder with `all-MiniLM-L6-v2` (22M parameters, 384-d), keeping the pipeline otherwise identical.

Result: all-MiniLM achieves MCC = 0.651 and AP = 0.657 — a drop of 7 MCC points and 8 AP points. Search Precision@10 falls from 0.565 to 0.352. The legal-domain advantage is substantial and consistent across metrics.

Next question: the base Legal-BERT was pre-trained only with masked language modelling. Can SBERT-style contrastive fine-tuning further improve the embedding quality?

2.3 SBERT fine-tuning: Legal-BERT-Embedding

Hypothesis: fine-tuning the encoder with a contrastive (Siamese) objective produces cosine-optimised sentence embeddings that improve both search and classification, without changing the inference architecture.

Setup: add `maticzav/legal-bert-embedding` — the same Legal-BERT backbone fine-tuned as a sentence-transformers model — to the model registry. Same pipeline, same 768 dimensions, same inference cost.

Result:

Model	Params	Dim	Build (CPU)	Val MCC	Val AP
Legal-BERT	110M	768	$\sim$21 min	0.720	0.736
Legal-BERT-Embedding	110M	768	$\sim$28 min	**0.764**	**0.742**
all-MiniLM-L6-v2	22M	384	$\sim$1.5 min	0.651	0.657

Legal-BERT-Embedding improves MCC by 4.4 points and search Precision@10 from 0.565 to 0.617 (a 26.9$\times$ gain over random). However, the AP gain is only 0.6 points (0.736 $\rightarrow$ 0.742).

Next question: is the 0.6-point AP improvement statistically significant, or just noise?

2.4 Significance: bootstrap analysis

Hypothesis: the SBERT fine-tuning genuinely improves ranking quality (AP), not just threshold-dependent metrics like MCC.

Setup: paired bootstrap (2,000 iterations, 95% CI) on the macro AP difference between model pairs.

Result: the AP difference between Legal-BERT-Embedding and base Legal-BERT is $\Delta = +0.006$, CI $[-0.038, +0.061]$ — the interval crosses zero, so the difference is **not statistically significant**. The MCC gain is therefore driven by better probability calibration enabling superior threshold selection, not by fundamentally better ranking. In contrast, the AP gap between either legal-domain model and all-MiniLM ($\sim$0.08, CI $[+0.019, +0.158]$) **is** significant.

Next question: how does the system compare to the published CLAUDETTE results [3]?

2.5 External validation: leave-one-document-out

Hypothesis: under the same evaluation protocol used by Lippi et al. (50-fold leave-one-document-out), our frozen-embedding approach is competitive with their hand-crafted feature ensemble.

Setup: replicate the LOO protocol: for each of the 50 documents, train on the other 49 (with fixed $C = 50$; our earlier sweep shows MCC varies by less than 0.01 for all $C > 10$), test on the held-out document. Train 8 independent Logistic Regressions on unfair-only sentences for categorization, matching Lippi's architectural setup.

Result:

	Detection F1	Categorization macro F1
Lippi et al. (ensemble / 8 SVMs)	0.806	$\sim$0.88
Our Legal-BERT + multi-LR / 8-LR	0.742 / 0.753	0.889

On detection, our single multi-output classifier (trained on all 9,414 sentences, fair and unfair) trails their 5-model ensemble (F1 0.75 vs. 0.81). This gap is partly due to model capacity (one linear model vs. an ensemble with complementary feature types) and partly due to the joint detection+categorisation task being harder than detection alone. On categorization, where the 8 independent classifiers are trained **only on unfair sentences** (matching Lippi's setup: each classifier learns to distinguish its category from other unfair categories, never seeing fair sentences), our approach is competitive (0.889 vs. $\sim$0.88) despite being dramatically simpler (linear classifiers on dense embeddings vs. SVMs with hand-crafted features). Per-category F1 ranges from 0.77 (Content Removal) to 0.94 (Choice of Law, Jurisdiction).

3 System Architecture

Informed by the experiments above, the service uses Legal-BERT-Embedding as the default encoder (best MCC and search quality at identical inference cost to base Legal-BERT; all-MiniLM is available for faster builds).

Components. The encoder produces a 768-d vector per sentence via mean pooling. FAISS `IndexFlatIP` stores all 9,414 vectors (exact search, $\sim$1–2 ms on CPU). A multi-output Logistic Regression classifier ($C = 50$, `class_weight='balanced'`) with per-category MCC-optimal thresholds predicts the eight unfairness categories from the same frozen embeddings. (The production system is single-stage: one classifier trained on all sentences handles both detection and categorisation simultaneously. For the Lippi comparison in §2.5 we also train 8 independent classifiers on unfair-only sentences — same model architecture, harder training data — to match their two-stage protocol.)

API. FastAPI with four endpoints: `/health` (GET), `/search` (POST), `/classify` (POST), `/analyze` (POST, combined). Pydantic validation (1–2000 characters, `top_k` 1–50). Models load once at startup; no network calls at inference.

Key design decisions.

Shared frozen encoder. Using the same encoder for search and classification eliminates an extra model dependency. The frozen weights make the classifier embedding-agnostic — swapping encoders requires only rebuilding the FAISS index.

Mean pooling over [CLS .] Reimers & Gurevych [5] showed mean pooling of hidden states produces substantially better sentence embeddings than the [CLS] token for semantic similarity.

Exact search. At 9,414 vectors, `IndexFlatIP` is both fast and precise. Approximate indices are unnecessary at this scale but remain a one-line change for larger corpora.

MCC as primary metric. MCC accounts for all four confusion-matrix quadrants and is unbiased by class imbalance ($\sim$11% positive). AP (threshold-agnostic) with bootstrap CIs is used to assess whether improvements are genuine or threshold artifacts. F1 is reported only for Lippi comparability.

Testing. Property-based tests (Hypothesis [4]) verify API invariants ($\forall$ text, `/classify` returns exactly 8 categories; all scores $\in [0, 1]$). 715 test cases pass. Conventional pytest covers the remaining cases.

4 Limitations & Trade-offs

Frozen embeddings. Keeping the encoder frozen enables direct model comparison but leaves task-specific structure unexploited. Contrastive fine-tuning with the eight labels could further improve both search and classification.

Evaluation protocol. Our LOO implementation uses a fixed $C = 50$ rather than retuning per fold. The MCC plateau above $C = 10$ makes this harmless in practice, but a fully nested protocol would be more rigorous. The categorization LOO was run only for Legal-BERT; cross-model LOO comparison remains future work.

Class imbalance. At $\sim$11% positive and per-category rates as low as 2–3%, rare categories (especially Arbitration) show high variance. We mitigate with balanced class weights, stratified splits, and per-category MCC-optimal thresholds.

Production gaps. The API is stateless and validates inputs but lacks authentication, rate limiting, and persistent logging.

5 Running the Service

Docker (recommended). The service and UI run with a single command:

```
docker compose up --build
```

This builds Legal-BERT-Embedding inside the image ($\sim$28 min on CPU, $\sim$1.5 min with `all-minilm`). To use a different encoder:

```
docker compose build --build-arg MODEL=all-minilm
docker compose up
```

The API listens on port 8000, the Streamlit UI on port 8501.

Development (uv[1]). For local development or reproducing individual experiments:

```
# Install
uv sync --extra dev
```

[1] `https://docs.astral.sh/uv`

```
# Build and train (default: legal-bert-embedding, ~28 min on CPU)
uv run python -m src.build_models
uv run python -m src.train_classifier

# Alternative encoders
uv run python -m src.build_models --model all-minilm          # ~1.5 min
uv run python -m src.build_models --model legal-bert-embedding  # ~28 min
uv run python -m src.train_classifier --model <model-key>

# Evaluation scripts (reproduce every number in this report)
uv run python -m src.evaluate_search --model <model-key>
uv run python -m src.bootstrap_ap --n-bootstrap 2000
uv run python -m src.evaluate_loo --model legal-bert --categorization

# Service + UI
uv run python -m src.service       # API on port 8000
uv run python -m src.ui            # Streamlit on port 8501
```

References

[1] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. LEGAL-BERT: The muppets straight out of law school. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020.

[2] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[3] Marco Lippi, Przemysław Pałka, Giuseppe Contissa, Francesca Lagioia, Hans-Wolfgang Micklitz, Giovanni Sartor, and Paolo Torroni. CLAUDETTE: an automated detector of potentially unfair clauses in online terms of service. *Artificial Intelligence and Law*, 27(2):117–139, 2019.

[4] David MacIver and Contributors. Hypothesis: Property-based testing for Python. `https://hypothesis.readthedocs.io/`, 2024.

[5] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.